Programmatic Modeling of High-Order Observers for Labeled Finite-State Automata

Zixi Wang

Contents

1 Introduction **2**

2 Regular Language and FAs **2**
2.1 Alphabets, words, and languages 2
2.2 Regular languages 3
2.3 Deterministic and nondeterministic finite automata 4

3 LFSAs **5**

4 Concurrent Composition **6**
4.1 Definition 6
4.2 Physical Meaning 7
4.3 Applications 7

5 Observer **8**
5.1 Definition 8
5.2 Physical Meaning 9
5.3 Applications 9

6 High-order observer **10**
6.1 Model of Agent 10
6.2 Definition 11
6.3 Physical Meaning 11
6.4 Applications 11
6.5 Pipeline to get the high-order observer 12

7 Future Work **14**

References **14**

1 Introduction

Consider a finite-state system G observed by two agents. Agent A_1 estimates the current state of G from its observations. Based on its own observations, Agent A_2 infers the possible state estimates held by Agent A_1.

This article develops a programmatic pipeline for constructing the order-2 observer, which computes Agent A_2's inference about Agent A_1's state estimate.

The following sections introduce the theoretical foundations and constructions used to implement this pipeline.

2 Regular Language and FAs

The following definitions summarize the background needed in this project.

2.1 Alphabets, words, and languages

An **alphabet** is a finite, nonempty set of letters, usually denoted by Σ. A **word** over Σ is a finite sequence of letters from Σ. More precisely, a word of length n has the form

$$w = a_1 a_2 \cdots a_n \qquad a_i \in \Sigma.$$

The empty word is denoted by ϵ. For every $n \in \mathbb{N}$,

$$\Sigma^0 = \{\epsilon\}, \qquad \Sigma^n = \{a_1 a_2 \cdots a_n \mid a_i \in \Sigma,\ 1 \le i \le n\} \quad (n \ge 1).$$

The **Kleene star**

$$\Sigma^* = \bigcup_{n \ge 0} \Sigma^n = \{\epsilon\} \cup \Sigma \cup \Sigma^2 \cup \cdots$$

is the set of all finite words over Σ. A **formal language over** Σ is any set

$$L \subseteq \Sigma^*.$$

Given $L_1, L_2 \subseteq \Sigma^*$, there are several basic operations:

$$\begin{aligned} L_1 \cup L_2 &= \{w \in \Sigma^* \mid w \in L_1 \text{ or } w \in L_2\}, \\ L_1 \cap L_2 &= \{w \in \Sigma^* \mid w \in L_1 \text{ and } w \in L_2\}, \\ \overline{L_1} &= \Sigma^* \setminus L_1, \\ L_1 \setminus L_2 &= \{w \in L_1 \mid w \notin L_2\}, \\ L_1 L_2 &= \{uv \mid u \in L_1,\ v \in L_2\}. \end{aligned}$$

For a language L, define $L^0 = \{\epsilon\}$ and $L^{n+1} = L^n L$. Its **Kleene star** is

$$L^* = \bigcup_{n \ge 0} L^n.$$

2.2 Regular languages

A **regular expression over** Σ is defined recursively. The expressions $\emptyset$, ϵ, and a for every $a \in \Sigma$ are regular expressions. If r and s are regular expressions, then

$$(r), \qquad r \cup s, \qquad rs, \qquad r^*$$

are also regular expressions, representing parentheses, union, concatenation, and Kleene star, respectively.

The language denoted by a regular expression r, written $L(r)$, is defined recursively by

$$\begin{aligned} L(\emptyset) &= \emptyset, & L(\epsilon) &= \{\epsilon\}, & L(a) &= \{a\}, \\ L(r \cup s) &= L(r) \cup L(s), & L(rs) &= L(r)L(s), & L(r^*) &= L(r)^*. \end{aligned}$$

A language $L \subseteq \Sigma^*$ is **regular** if and only if there exists a regular expression r over Σ such that

$$L = L(r).$$

Equivalently, L is regular if and only if it is recognized by a finite automaton.

Regular languages are closed under union, intersection, complement, difference, concatenation, and Kleene star. In other words, if L_1 and L_2 are regular, then

$$L_1 \cup L_2, \quad L_1 \cap L_2, \quad \overline{L_1}, \quad L_1 \setminus L_2, \quad L_1 L_2, \quad L_1^*$$

are also regular.

Let a DFA $A = (Q, \Sigma, \delta, q_0, F)$ recognize the regular language L, and choose the **pumping length**

$$p = |Q|.$$

For every $w \in L$ with $|w| \geq p$, the run of A while reading the first p letters visits a state more than once. Therefore,

$$\forall w \in L \text{ with } |w| \geq p, \ \exists x, y, z \in \Sigma^*$$

such that

- $w = xyz$;
- $|xy| \leq p$;
- $y \neq \epsilon$;
- $\forall i \in \mathbb{N}$, $xy^i z \in L$.

Thus, the number of states of a DFA recognizing L is a valid pumping length, although it is not necessarily the smallest possible pumping length.

2.3 Deterministic and nondeterministic finite automata

A deterministic finite automaton (DFA) is a quintuple

$$A = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q is a finite set of states;
- Σ is the input alphabet;
- $\delta : Q \times \Sigma \to Q$ is the transition function;
- $q_0 \in Q$ is the initial state;
- $F \subseteq Q$ is the set of final states.

The transition function extends from letters to words. For $q \in Q$, $w \in \Sigma^*$, and $a \in \Sigma$,

$$\delta(q, \epsilon) = q, \qquad \delta(q, wa) = \delta(\delta(q, w), a).$$

The DFA accepts w exactly when $\delta(q_0, w) \in F$. Its recognized language is

$$L(A) = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}.$$

A nondeterministic finite automaton (NFA) is a quintuple

$$N = (Q, \Sigma, \delta, Q_0, F),$$

where:

- Q is a finite set of states;
- Σ is the input alphabet;
- $\delta : Q \times \Sigma \to 2^Q$ is the transition function;
- $Q_0 \subseteq Q$ is the set of initial states;
- $F \subseteq Q$ is the set of final states.

Here, $2^Q = \{X \mid X \subseteq Q\}$ is the power set of Q, so $\delta(q, a)$ may contain zero, one, or several successor states. Extend δ to sets of states and words by

$$\delta(X, \epsilon) = X, \qquad \delta(X, wa) = \bigcup_{q \in \delta(X, w)} \delta(q, a),$$

for $X \subseteq Q$, $w \in \Sigma^*$, and $a \in \Sigma$. The NFA accepts a word w if at least one state reachable from an initial state is accepting:

$$\delta(Q_0, w) \cap F \neq \emptyset.$$

Therefore, the language recognized by N is

$$L(N) = \{w \in \Sigma^* \mid \delta(Q_0, w) \cap F \neq \emptyset\}.$$

Every NFA can be converted into an equivalent DFA by the **subset construction**.

Kleene's theorem establishes the equivalence

$$\text{regular expressions} \iff \text{NFAs} \iff \text{DFAs}.$$

In this project, the underlying system is usually written as

$$G = (Q, E, \delta, Q_0),$$

where E is an event alphabet, $\delta \subseteq Q \times E \times Q$ is a transition relation, and $Q_0 \subseteq Q$ is a set of initial states. Accepting states are omitted because the project studies generated runs and state estimates rather than language acceptance. This transition-system form may be nondeterministic and may have multiple initial states. Adding an observation alphabet and a labeling function to G produces the LFSA introduced in the next section.

3 LFSAs

According to Definition 1 in Zhang (2023), a **labeled finite-state automaton (LFSA)** is a sextuple

$$\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell),$$

where:

- Q is a finite set of states;
- E is a finite alphabet of events;
- $\delta : Q \times E \to 2^Q$ is the transition function;
- $Q_0 \subseteq Q$ is the set of initial states;
- Σ is a finite alphabet of observable labels;
- $\ell : E \to \Sigma \cup \{\epsilon\}$ is the labeling function.

A transition $q' \in \delta(q, e)$, equivalently $(q, e, q') \in \delta$, means that the occurrence of event e moves the LFSA from state q to state q'. Since $\delta(q, e)$ is a set of states, an LFSA may be nondeterministic.

The event alphabet is partitioned into the observable and unobservable event sets

$$E_o = \{e \in E \mid \ell(e) \in \Sigma\}, \qquad E_{uo} = \{e \in E \mid \ell(e) = \epsilon\},$$

When an event in E_o occurs, its label is observed. When an event in E_{uo} occurs, the observer receives no output.

The transition function is extended from events to finite event sequences. For $q \in Q$, $s \in E^*$, and $e \in E$,

$$\delta(q, \epsilon) = \{q\}, \qquad \delta(q, se) = \bigcup_{p \in \delta(q,s)} \delta(p, e).$$

For $X \subseteq Q$, define

$$\delta(X, s) = \bigcup_{q \in X} \delta(q, s).$$

The labeling function is extended from events to event sequences by:

$$\ell(\epsilon) = \epsilon, \qquad \ell(e_1 e_2 \cdots e_n) = \ell(e_1)\ell(e_2)\cdots\ell(e_n).$$

The set of finite event traces generated by $\mathcal{S}$ is

$$L(\mathcal{S}) = \{s \in E^* \mid \delta(Q_0, s) \neq \emptyset\}.$$

The observable language generated by $\mathcal{S}$ is

$$\mathcal{L}(\mathcal{S}) = \{\ell(s) \in \Sigma^* \mid s \in L(\mathcal{S})\}.$$

For an observed word $\sigma \in \Sigma^*$, the **current-state estimate** is

$$\mathcal{M}(\mathcal{S}, \sigma) = \{q \in Q \mid \exists q_0 \in Q_0,\ \exists s \in E^* : \ell(s) = \sigma \text{ and } q \in \delta(q_0, s)\}.$$

Thus, an LFSA distinguishes an internal event sequence s from its observable label sequence $\ell(s)$. Different event sequences may generate the same observed word, which is the source of state uncertainty studied by concurrent composition and observers.

4 Concurrent Composition

4.1 Definition

According to Definition 2 in Zhang (2023), consider two LFSAs with the same output alphabet Σ:

$$\mathcal{S}_i = (Q_i, E_i, \delta_i, Q_{0i}, \Sigma, \ell_i), \qquad i \in \{1, 2\}.$$

For each $\mathcal{S}_i$, define

$$E_{io} = \{e \in E_i \mid \ell_i(e) \in \Sigma\}, \qquad E_{iuo} = \{e \in E_i \mid \ell_i(e) = \epsilon\}.$$

Their **concurrent composition** is the LFSA

$$\mathrm{CC}_A(\mathcal{S}_1, \mathcal{S}_2) = (Q', E', \delta', Q'_0, \Sigma, \ell'),$$

whose components are defined as follows.

1. The state set is the Cartesian product

$$Q' = Q_1 \times Q_2.$$

   Thus, a state (q_1, q_2) records the states currently occupied by the two component LFSAs.

2. The initial-state set is

$$Q'_0 = Q_{01} \times Q_{02}.$$

3. The observable events synchronize events having the same label:

$$E'_o = \{(e_1, e_2) \in E_{1o} \times E_{2o} \mid \ell_1(e_1) = \ell_2(e_2)\}.$$

4. The unobservable events occur independently in either component:

$$E'_{uo} = \{(e_1, \epsilon) \mid e_1 \in E_{1uo}\} \cup \{(\epsilon, e_2) \mid e_2 \in E_{2uo}\}.$$

   Therefore,

$$E' = E'_o \cup E'_{uo}.$$

5. For a synchronized observable event $(e_1, e_2) \in E'_o$,

$$((q_1, q_2), (e_1, e_2), (q'_1, q'_2)) \in \delta' \iff (q_1, e_1, q'_1) \in \delta_1 \text{ and } (q_2, e_2, q'_2) \in \delta_2.$$

6. For a left-component unobservable event $(e_1, \epsilon) \in E'_{uo}$,

$$((q_1, q_2), (e_1, \epsilon), (q'_1, q'_2)) \in \delta' \iff (q_1, e_1, q'_1) \in \delta_1 \text{ and } q_2 = q'_2.$$

7. For a right-component unobservable event $(\epsilon, e_2) \in E'_{uo}$,

$$((q_1, q_2), (\epsilon, e_2), (q'_1, q'_2)) \in \delta' \iff q_1 = q'_1 \text{ and } (q_2, e_2, q'_2) \in \delta_2.$$

8. The labeling function is

$$\ell'((e_1, e_2)) = \ell_1(e_1) = \ell_2(e_2),$$

   where the labeling functions are extended by $\ell_1(\epsilon) = \ell_2(\epsilon) = \epsilon$.

If $\mathcal{S}_1 = \mathcal{S}_2 = \mathcal{S}$, then

$$\mathrm{CC}_A(\mathcal{S}, \mathcal{S}) = \mathrm{CC}_A(\mathcal{S})$$

is called the **self-composition** of $\mathcal{S}$.

4.2 Physical Meaning

Consequently, observable transitions can occur together only when they produce the same label, whereas unobservable transitions can occur separately without changing the other component's state. The concurrent composition therefore collects pairs of runs from $\mathcal{S}_1$ and $\mathcal{S}_2$ that generate the same observable label sequence.

4.3 Applications

This interpretation makes concurrent composition a general verification tool for discrete-event systems. By pairing runs that are indistinguishable under observation, it reduces questions about detectability, diagnosability, opacity, and related state/event inference or concealment properties to checking property-specific structural conditions on the reachable part of the composed system.

Example—strong detectability. After sufficiently many observations, is the current state uniquely known? Self-composition tracks pairs of indistinguishable runs. If a reachable cycle contains a state (q_1, q_2) with $q_1 \neq q_2$, ambiguity can persist indefinitely; therefore, strong detectability fails.

Example—fault diagnosability. Can a fault always be confirmed after finitely many observations? Concurrent composition pairs a faulty run with a fault-free run that produces the same observation. If a reachable cycle maintains such a pair, the ambiguity can persist indefinitely; therefore, the fault is not diagnosable.

Example—current-state opacity. Can an observer determine that the current state is secret? Suppose a secret state q_s and a non-secret state q_n can both generate the same observation a. Concurrent composition tracks the indistinguishable pair (q_s, q_n). Because the observation is consistent with a non-secret state, the secret remains opaque. If no such non-secret counterpart is reachable, current-state opacity fails.

Example—event inference. Can an observer determine whether a particular event has occurred? Concurrent composition pairs a run containing the event with a run not containing it but producing the same observation. If such a pair is reachable, the event cannot be inferred uniquely; otherwise, its occurrence is identifiable.

Example—secret-event opacity. Can the occurrence of a secret event be concealed? Concurrent composition pairs a run containing the secret event with a run containing no secret event but producing the same observation. If every secret run has such an indistinguishable non-secret counterpart, the secret event remains opaque; if a secret run has none, opacity fails.

5 Observer

5.1 Definition

According to Definition 3 in Zhang (2023), consider an LFSA

$$\mathcal{S} = (Q, E, \delta, Q_0, \Sigma, \ell)$$

with observable event set E_o and unobservable event set E_{uo}. Its **observer** is the deterministic finite automaton

$$\mathcal{S}_{\text{obs}} = (Q_{\text{obs}}, \Sigma_{\text{obs}}, \delta_{\text{obs}}, q_{0\ \text{obs}}).$$

Each observer state is a set of possible states of the original LFSA. The components of the observer are defined as follows.

1. The observer state set is the power set

$$Q_{\text{obs}} = 2^Q.$$

2. The observer alphabet is the set of observable labels generated by events:

$$\Sigma_{\text{obs}} = \ell(E_o) = \ell(E) \setminus \{\epsilon\}.$$

3. For any $X \subseteq Q$, its **unobservable reach** is

$$\text{UR}(X) = \bigcup_{q \in X} \bigcup_{s \in E_{uo}^*} \delta(q, s).$$

4. The initial observer state contains every state reachable from an original initial state through only unobservable events:

$$q_{0\ \mathrm{obs}} = \mathrm{UR}(Q_0).$$

5. For $X \in Q_{\mathrm{obs}}$ and $a \in \Sigma_{\mathrm{obs}}$, first take every observable event labeled a, and then take the unobservable reach:

$$\delta_{\mathrm{obs}}(X, a) = \mathrm{UR}\left(\bigcup_{q \in X} \bigcup_{\substack{e \in E_o \\ \ell(e) = a}} \delta(q, e) \right).$$

Because $\delta_{\mathrm{obs}}(X, a)$ is one uniquely determined subset of Q, the observer is deterministic even when the original LFSA is nondeterministic. For every observed word $\sigma \in \Sigma_{\mathrm{obs}}^*$,

$$\delta_{\mathrm{obs}}(q_{0\ \mathrm{obs}}, \sigma) = \mathcal{M}(\mathcal{S}, \sigma).$$

5.2 Physical Meaning

Thus, after reading σ, the observer state is exactly the current-state estimate of the LFSA: it contains all and only the states in which $\mathcal{S}$ could be after producing σ. Although $Q_{\mathrm{obs}} = 2^Q$ contains all subsets theoretically, an implementation usually constructs only the subsets reachable from $q_{0\ \mathrm{obs}}$.

5.3 Applications

Observer applications include current-state estimation, detectability verification, fault diagnosis, opacity analysis, and state-based supervisory control under partial observation.

Example—current-state estimation. After observing a word σ, which states may the system currently occupy? Starting from its initial state, the observer follows the transitions labeled by σ. The reached observer state $\mathcal{M}(\mathcal{S}, \sigma)$ contains exactly all possible current states of the original system.

Example—detectability verification. Can the current state eventually be determined uniquely from observations? The observer verifies this by examining its reachable state estimates. Persistent reachable estimates containing multiple states reveal observational ambiguity and, depending on the chosen detectability notion, may violate detectability.

Example—fault diagnosis. Can the occurrence of an unobservable fault be determined after finitely many observations? Using states augmented with fault information, the observer tracks fault-free and faulty possibilities. An estimate containing both possibilities is uncertain; persistent uncertainty means that the fault is not diagnosable.

Note on fault diagnosis.

1. *What is a fault?* A fault is an event, usually unobservable, representing equipment or component failure.

2. *What is a fault-augmented system?* It combines the plant with a memory that records whether a fault has occurred.
    - *What mathematical structure is used to describe the memory?* A two-state automaton $\{N, F\}$, where $N \xrightarrow{f} F$ and F remains F.
3. *What are plant states and fault statuses?* The plant state q is the current system state, while N or F records whether a fault has occurred. Thus, an augmented state is (q, N) or (q, F).

Example—opacity analysis. Can an external observer determine that the system is in a secret state? Let $Q_S \subseteq Q$ be the secret-state set. If a reachable observer state X satisfies $X \subseteq Q_S$, the observation reveals the secret; therefore, current-state opacity fails.

Example—state-based supervisory control. Which controllable events can be enabled safely under partial observation? For each observer state, a supervisor considers every possible current state represented by that estimate and enables only decisions that satisfy the control requirement for all of them.

Note on state-based supervisory control.

- *What is a controllable event?* The event set is partitioned as $E = E_c \dot\cup E_{uc}$. Events in E_c may be disabled by the supervisor, whereas events in E_{uc} cannot.
- *What is a forbidden state?* Forbidden states form a predefined set $Q_{\text{bad}} \subseteq Q$ containing states that the controlled system must avoid.

6 High-order observer

6.1 Model of Agent

Consider an FSA

$$G = (Q, E, \delta, Q_0).$$

An **agent** A_i is modeled by the triple

$$A_i = (E_i, \Sigma_i, \ell_i),$$

where:

- $E_i \subseteq E$ is the set of events observable to A_i;
- Σ_i is the finite alphabet of labels observed by A_i;
- $\ell_i : E_i \to \Sigma_i$ is the labeling function of A_i.

The labeling function is extended to $\bar{\ell}_i : E \to \Sigma_i \cup \{\epsilon\}$ by

$$\bar{\ell}_i(e) = \begin{cases} \ell_i(e), & e \in E_i, \\ \epsilon, & e \in E \setminus E_i. \end{cases}$$

For simplicity, the extended function $\bar{\ell}_i$ is also denoted by ℓ_i in the following. Thus, an event in E_i produces an observable label, while an event outside E_i is unobservable to A_i. Different observable events may have the same label.

For an event trace $s = e_1 e_2 \cdots e_n \in E^*$, the observation made by A_i is

$$\ell_i(s) = \ell_i(e_1)\ell_i(e_2)\cdots\ell_i(e_n) \in \Sigma_i^*,$$

where every occurrence of ϵ disappears under concatenation. Two event traces are indistinguishable to A_i if they produce the same observation:

$$s \sim_{A_i} t \quad \Longleftrightarrow \quad \ell_i(s) = \ell_i(t).$$

Combining the FSA G with the observation mechanism of A_i produces the LFSA

$$G_{A_i} = (Q, E, \delta, Q_0, \Sigma_i, \ell_i).$$

The agent does not change the states, events, or transitions of G; it only determines how the events are observed. Consequently, different agents may produce different LFSAs from the same FSA because they may have different observable event sets, alphabets, or labeling functions.

6.2 Definition

Consider an FSA

$$G = (Q, E, \delta, Q_0)$$

and an ordered sequence of agents $A_1, A_2, \ldots, A_n$. Agent A_i observes an event trace $s \in L(G)$ through its labeling function

$$\ell_i : E \to \Sigma_i \cup \{\epsilon\}.$$

An ordinary observer gives an agent's estimate of the current state of G. This is an **order-1 estimate**:

$$\mathcal{M}_{A_1}(G, \alpha) = \{q \in Q \mid \exists q_0 \in Q_0,\ \exists s \in L(G) : \ell_1(s) = \alpha \text{ and } q \in \delta(q_0, s)\} \in 2^Q.$$

A high-order estimate describes one agent's estimate of another agent's estimate. For example, after A_2 observes $\beta \in \ell_2(L(G))$, its order-2 estimate of what A_1 may know is

$$\mathcal{M}_{A_1 \leftarrow A_2}(G, \beta) = \{\mathcal{M}_{A_1}(G, \ell_1(s)) \mid s \in L(G),\ \ell_2(s) = \beta\} \in 2^{2^Q}.$$

6.3 Physical Meaning

Thus, A_1 estimates the current state of G, whereas A_2 estimates the possible state estimates held by A_1. The arrow in $A_1 \leftarrow A_2$ indicates that the final inference is made by A_2 about A_1.

6.4 Applications

High-order observers provide a systematic way to solve **epistemic inference problems** in discrete-event systems. They can determine what one agent knows about another agent's state estimate, verify higher-order opacity and information-flow privacy, analyze epistemic detectability and diagnosability, and support decentralized supervisory-control decisions that depend on nested knowledge.

Example—high-order opacity. Can agent A_2 determine that agent A_1 knows the system is in a secret state? The order-2 observer tracks all state estimates that A_1 may hold from A_2's viewpoint. If a reachable order-2 estimate contains only A_1-estimates that lie entirely within the secret-state set, then A_2 knows that A_1 knows the secret; therefore, high-order opacity fails.

Example—information-flow privacy. Can agent A_2 infer private information available to agent A_1? The high-order observer tracks the estimates that A_1 may hold after each observation of A_2. If a reachable high-order estimate reveals only one possible private value, then A_2 can infer that value; therefore, information-flow privacy fails.

Note on information-flow privacy.

- *What is private information?* It is an attribute of A_1's state estimate that should not be uniquely inferred by A_2, such as whether the estimate reveals a secret state, event, or control decision.
- *How is private information represented?* Let $h : 2^Q \to V$, where V is the set of possible private values; for example, $V = \{\text{secret}, \text{non-secret}\}$.
- *What does A_1 hide?* Agent A_1 hides the private attribute value $h(X)$ associated with its state estimate X, rather than necessarily hiding the state estimate itself.

Example—epistemic detectability. Can agent A_2 determine that agent A_1 knows the current state uniquely? The order-2 observer tracks all estimates that A_1 may hold from A_2's viewpoint. If every estimate in a reachable order-2 state is the same singleton, then A_2 knows that A_1 knows the current state.

Note on the same singleton. If a reachable order-2 state is $\{X_1, \ldots, X_k\}$, then "every estimate is the same singleton" means

$$X_1 = \cdots = X_k = \{q\}$$

for some $q \in Q$. Thus, all possible estimates held by A_1 agree on exactly one current state.

Example—epistemic diagnosability. Can agent A_2 determine that agent A_1 has diagnosed a fault? First augment the plant states with a fault status. The order-2 observer then tracks all augmented-state estimates that A_1 may hold. If every estimate in a reachable order-2 state contains only states marked as faulty, then A_2 knows that A_1 has diagnosed the fault.

Example—decentralized supervisory control. Can agent A_2 safely enable an event whose decision depends on what agent A_1 knows? The high-order observer collects all estimates that A_1 may hold from A_2's viewpoint. Agent A_2 enables the event only if the required control condition holds for every such estimate; otherwise, enabling it may violate the specification.

6.5 Pipeline to get the high-order observer

Consider an FSA $G = (Q, E, \delta, Q_0)$ and two ordered agents $A_1 = (E_1, \Sigma_1, \ell_1)$ and $A_2 = (E_2, \Sigma_2, \ell_2)$. An event outside E_i is unobservable to A_i, so its label under ℓ_i is ϵ. The order-2 observer is constructed as follows.

1. Combine G with A_1's observation mechanism to obtain the LFSA

$$G_{A_1} = (Q, E, \delta, Q_0, \Sigma_1, \ell_1).$$

   In the program, `src/fa_agent_to_lfsa.py` reads the FSA JSON and the A_1 JSON and writes the LFSA JSON.

2. Apply the powerset construction to G_{A_1}:

$$\mathrm{Obs}_{\ell_1}(G) =: \mathrm{Obs}_{A_1}.$$

   Each observer state $X \subseteq Q$ is A_1's current-state estimate after an observation. The program `src/observer.py` constructs the nonempty reachable part of this observer. Although the observer is mathematically a DFA, its JSON representation uses identity labels, $\ell_{\mathrm{obs}}(a) = a$, so it can be passed to the concurrent-composition program.

3. Concurrently compose G_{A_1} and Obs_{A_1}:

$$CC_{\ell_1}\left(G_{A_1}, \mathrm{Obs}_{A_1}\right).$$

   A state of this composition has the form (q, X), where q is the actual state of G and X is the state estimate held by A_1. Observable transitions synchronize under the same ℓ_1-label, while unobservable transitions of G_{A_1} interleave. This step is implemented by `src/CC.py`.

4. The composition is still labeled from A_1's perspective. To prepare it for A_2, first project every paired composition event

$$(e_1, e_2) \longmapsto e_1$$

   and remove the existing ℓ_1-labels. This produces the underlying FSA whose states remain pairs (q, X). The operation is provided by `src/lfsa_to_fa.py` using its `-project-left` option.

5. Combine this projected FSA with A_2. This replaces ℓ_1 with ℓ_2 and produces the LFSA $CC^{G,\mathrm{Obs}}_{A_1 \to A_2}$. In the program, run `src/fa_agent_to_lfsa.py` with the projected FSA and the A_2 agent JSON.

6. Apply `src/observer.py` to the A_2-labeled composition:

$$\mathrm{Obs}_{\ell_2}\left(CC^{G,\mathrm{Obs}}_{A_1 \to A_2}\right) =: \mathrm{Obs}_{A_1 \leftarrow A_2}(G).$$

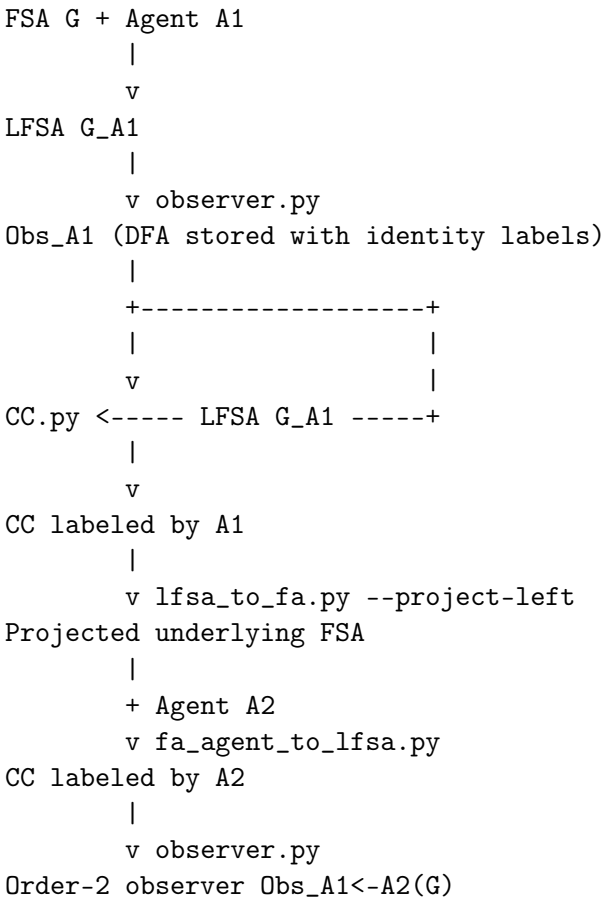
   This is the order-2 observer. A state has the form

$$\{(q_1, X_1), \ldots, (q_n, X_n)\}.$$

   After A_2 observes a word α, its possible inferences of A_1's estimate are

$$M_{A_1 \leftarrow A_2}(G, \alpha) = \{X_1, \ldots, X_n\}.$$

The programmatic data flow is therefore



For $n > 2$, the construction is repeated recursively: project the current composition onto the original event component, apply the next agent's labeling function, and construct another observer. Each new observer adds one level of state-estimate nesting. In the general case, constructing an order-n observer has n-EXPTIME complexity.

7 Future Work

Future work will focus on the following directions:

- automate the recursive construction of order-n observers;
- add verification procedures for high-order opacity, detectability, and diagnosability;
- develop benchmark examples and automated correctness tests;

References

[1] K. Zhang, "A unified concurrent-composition method to state/event inference and concealment in labeled finite-state automata as discrete-event systems," *Annual Reviews in Control*, vol. 56, article 100902, 2023. https://doi.org/10.1016/j.arcontrol.2023.100902.

[2] K. Zhang, X. Han, A. Giua, and C. Seatzu, "[Draft] High-order observers and high-order state-estimation-based properties of discrete-event systems," *arXiv preprint* arXiv:2408.06141, 2024. https://doi.org/10.48550/arXiv.2408.06141.